Contents

Complete DSA Concepts Guide **4**
Master Data Structures & Algorithms for Product-Based Company Interviews . . . 4

Table of Contents **4**

1. Time & Space Complexity **4**
Big O Notation . . . 4
Common Complexities (Best to Worst) . . . 4
Rules for Calculating Complexity . . . 5
Space Complexity . . . 5

2. Arrays **5**
Fundamentals . . . 5
Key Operations . . . 5
Important Techniques . . . 5
1. Two Pointers . . . 5
2. Prefix Sum . . . 6
3. Kadane's Algorithm (Maximum Subarray) . . . 6
4. Dutch National Flag (3-way partition) . . . 6

3. Strings **6**
Fundamentals . . . 6
Key Operations . . . 6
Important Techniques . . . 7
1. Character Frequency . . . 7
2. Palindrome Check . . . 7
3. Anagram Check . . . 7
4. KMP Algorithm (Pattern Matching) . . . 7

4. Linked Lists **8**
Types . . . 8
Node Structure . . . 8
Key Operations . . . 8
Important Techniques . . . 8
1. Reverse Linked List . . . 8
2. Fast and Slow Pointers (Floyd's) . . . 9
3. Merge Two Sorted Lists . . . 9

5. Stacks **9**
Fundamentals . . . 9
Applications . . . 10
Important Techniques . . . 10
1. Valid Parentheses . . . 10
2. Monotonic Stack (Next Greater Element) . . . 10
3. Largest Rectangle in Histogram . . . 10

6. Queues **11**
Fundamentals . . . 11
Types . . . 11
Applications . . . 11
Sliding Window Maximum (Monotonic Deque) . . . 11

7. Hash Tables **11**
Fundamentals . . . 11
Applications . . . 12
Two Sum . . . 12

Subarray Sum Equals K . . . 12

8. Trees **12**
Binary Tree . . . 12
Tree Traversals . . . 13
DFS Traversals . . . 13
Iterative Inorder . . . 13
BFS (Level Order) . . . 13
Binary Search Tree (BST) . . . 13
Important Tree Algorithms . . . 14
Height/Depth . . . 14
Lowest Common Ancestor . . . 14
Diameter of Tree . . . 14

9. Heaps **15**
Fundamentals . . . 15
Applications . . . 15
Kth Largest Element . . . 15
Merge K Sorted Lists . . . 15
Median from Data Stream . . . 16

10. Graphs **16**
Representations . . . 16
Adjacency List . . . 16
Adjacency Matrix . . . 16
Graph Traversals . . . 17
BFS . . . 17
DFS . . . 17
Important Graph Algorithms . . . 17
Topological Sort (Kahn's Algorithm) . . . 17
Cycle Detection (Directed Graph) . . . 18
Dijkstra's Algorithm (Shortest Path) . . . 18
Bellman-Ford (Negative Weights) . . . 18
Number of Islands (Grid BFS/DFS) . . . 18

11. Sorting Algorithms **19**
Comparison of Sorting Algorithms . . . 19
Implementations . . . 19
Merge Sort . . . 19
Quick Sort . . . 20
Counting Sort . . . 20

12. Searching Algorithms **20**
Binary Search . . . 20
Binary Search Variants . . . 21
Binary Search on Answer . . . 21

13. Recursion **22**
Fundamentals . . . 22
Recursion Patterns . . . 22
1. Linear Recursion . . . 22
2. Binary Recursion (Divide & Conquer) . . . 22
3. Tail Recursion . . . 22

14. Dynamic Programming **22**
When to Use DP . . . 22
Approaches . . . 23

Top-Down (Memoization) . . . 23
Bottom-Up (Tabulation) . . . 23
Classic DP Problems . . . 23
Climbing Stairs . . . 23
Coin Change . . . 23
Longest Common Subsequence . . . 23
0/1 Knapsack . . . 24
Longest Increasing Subsequence . . . 24
Edit Distance . . . 24

15. Greedy Algorithms **25**
When to Use . . . 25
Classic Problems . . . 25
Activity Selection . . . 25
Jump Game . . . 25
Gas Station . . . 25

16. Backtracking **26**
Template . . . 26
Classic Problems . . . 26
Subsets . . . 26
Permutations . . . 26
N-Queens . . . 26

17. Bit Manipulation **27**
Basic Operations . . . 27
Common Tricks . . . 27
Single Number (XOR) . . . 28

18. Two Pointers **28**
Patterns . . . 28
1. Opposite Direction . . . 28
2. Same Direction (Fast/Slow) . . . 28
3. Container With Most Water . . . 29

19. Sliding Window **29**
Fixed Size Window . . . 29
Variable Size Window . . . 29
Longest Substring Without Repeating . . . 29

20. Tries **30**
Structure . . . 30

21. Union Find (Disjoint Set) **30**
Structure . . . 30
Applications . . . 31

22. Important Patterns **31**
1. Prefix Sum . . . 31
2. Monotonic Stack . . . 31
3. BFS for Shortest Path . . . 31
4. DFS for Exploration . . . 31
5. Binary Search on Answer . . . 31
6. Interval Problems . . . 31
7. Cycle Detection . . . 31
8. Tree DP . . . 32

Quick Reference Card **32**

Time Complexities 32
Algorithm Complexities 32

Complete DSA Concepts Guide

Master Data Structures & Algorithms for Product-Based Company Interviews

---

Table of Contents

1. Time & Space Complexity
2. Arrays
3. Strings
4. Linked Lists
5. Stacks
6. Queues
7. Hash Tables
8. Trees
9. Heaps
10. Graphs
11. Sorting Algorithms
12. Searching Algorithms
13. Recursion
14. Dynamic Programming
15. Greedy Algorithms
16. Backtracking
17. Bit Manipulation
18. Two Pointers
19. Sliding Window
20. Tries
21. Union Find
22. Important Patterns

---

1. Time & Space Complexity

Big O Notation

Big O describes the upper bound of time/space as input grows.

Common Complexities (Best to Worst)

Notation	Name	Example
O(1)	Constant	Array access, Hash lookup
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, Single loop
O(n log n)	Linearithmic	Merge sort, Quick sort
O(n^2)	Quadratic	Nested loops, Bubble sort
O(n^3)	Cubic	Triple nested loops
O(2^n)	Exponential	Recursive Fibonacci
O(n!)	Factorial	Permutations

Rules for Calculating Complexity

1. **Drop constants:** O(2n) -> O(n)
2. **Drop lower terms:** O(n^2 + n) -> O(n^2)
3. **Different inputs, different variables:** O(a + b) not O(n)
4. **Consecutive operations add:** O(n) + O(m) = O(n + m)
5. **Nested operations multiply:** O(n) x O(m) = O(n x m)

Space Complexity

- **Input space:** Space used by input
- **Auxiliary space:** Extra space used by algorithm
- **Total space = Input + Auxiliary**

```python
# O(n) space - creates new array
def double_array(arr):
    return [x * 2 for x in arr]

# O(1) space - modifies in place
def double_in_place(arr):
    for i in range(len(arr)):
        arr[i] *= 2
```

---

2. Arrays

Fundamentals

- **Definition:** Contiguous memory locations storing elements of same type
- **Access:** O(1) by index
- **Search:** O(n) unsorted, O(log n) sorted
- **Insert/Delete:** O(n) due to shifting

Key Operations

```python
# Initialization
arr = [1, 2, 3, 4, 5]
arr = [0] * n  # Array of n zeros
matrix = [[0] * cols for _ in range(rows)]  # 2D array

# Common operations
arr.append(x)      # O(1) amortized
arr.pop()          # O(1)
arr.insert(i, x)   # O(n)
arr.remove(x)      # O(n)
arr[i]             # O(1)
len(arr)           # O(1)
```

Important Techniques

1. Two Pointers

```python
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        current = arr[left] + arr[right]
        if current == target:
            return [left, right]
```

```
        elif current < target:
            left += 1
        else:
            right -= 1
    return []
```

2. Prefix Sum

```
def prefix_sum(arr):
    prefix = [0] * (len(arr) + 1)
    for i in range(len(arr)):
        prefix[i + 1] = prefix[i] + arr[i]
    return prefix

# Sum of range [i, j] = prefix[j+1] - prefix[i]
```

3. Kadane's Algorithm (Maximum Subarray)

```
def max_subarray(arr):
    max_sum = current_sum = arr[0]
    for num in arr[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum
```

4. Dutch National Flag (3-way partition)

```
def sort_colors(arr):
    low, mid, high = 0, 0, len(arr) - 1
    while mid <= high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 1:
            mid += 1
        else:
            arr[mid], arr[high] = arr[high], arr[mid]
            high -= 1
```

---

3. Strings

Fundamentals

- Strings are immutable in Python/Java
- Character operations: `ord('a')` = 97, `chr(97)` = 'a'

Key Operations

```
s = "hello"
s[0]            # 'h' - O(1)
s[1:4]          # 'ell' - O(k)
s + " world"    # O(n + m) - creates new string
len(s)          # O(1)
s.find("ll")    # O(n*m)
s.replace()     # O(n)
```

Important Techniques

1. Character Frequency

```python
from collections import Counter
freq = Counter(s)  # {'h': 1, 'e': 1, 'l': 2, 'o': 1}

# Using array for lowercase letters
freq = [0] * 26
for c in s:
    freq[ord(c) - ord('a')] += 1
```

2. Palindrome Check

```python
def is_palindrome(s):
    return s == s[::-1]

# Two pointer approach
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True
```

3. Anagram Check

```python
def is_anagram(s1, s2):
    return Counter(s1) == Counter(s2)
    # Or: return sorted(s1) == sorted(s2)
```

4. KMP Algorithm (Pattern Matching)

```python
def compute_lps(pattern):
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length != 0:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    return lps

def kmp_search(text, pattern):
    lps = compute_lps(pattern)
    i = j = 0
    while i < len(text):
        if pattern[j] == text[i]:
            i += 1
            j += 1
```

```
        if j == len(pattern):
            return i - j  # Pattern found
        elif i < len(text) and pattern[j] != text[i]:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return -1
```

---

4. Linked Lists

Types

1. **Singly Linked List:** Each node points to next
2. **Doubly Linked List:** Each node points to next and prev
3. **Circular Linked List:** Last node points to first

Node Structure

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class DoublyListNode:
    def __init__(self, val=0, prev=None, next=None):
        self.val = val
        self.prev = prev
        self.next = next
```

Key Operations

Operation	Time
Access	O(n)
Search	O(n)
Insert at head	O(1)
Insert at tail	O(1) with tail pointer
Delete	O(1) if node given

Important Techniques

1. Reverse Linked List

```
def reverse(head):
    prev = None
    curr = head
    while curr:
        next_temp = curr.next
        curr.next = prev
        prev = curr
        curr = next_temp
    return prev
```

2. Fast and Slow Pointers (Floyd's)

```python
# Find middle
def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

# Detect cycle
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False

# Find cycle start
def find_cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            slow = head
            while slow != fast:
                slow = slow.next
                fast = fast.next
            return slow
    return None
```

3. Merge Two Sorted Lists

```python
def merge(l1, l2):
    dummy = ListNode(0)
    curr = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            curr.next = l1
            l1 = l1.next
        else:
            curr.next = l2
            l2 = l2.next
        curr = curr.next
    curr.next = l1 or l2
    return dummy.next
```

---

5. Stacks

Fundamentals

- **LIFO:** Last In, First Out

- All operations O(1)

```
stack = []
stack.append(x)  # Push
stack.pop()      # Pop
stack[-1]        # Peek
len(stack) == 0  # Is empty
```

Applications

1. **Balanced Parentheses**
2. **Expression Evaluation**
3. **Monotonic Stack**
4. **Function Call Stack**
5. **Undo Operations**

Important Techniques

1. Valid Parentheses

```python
def is_valid(s):
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for char in s:
        if char in mapping:
            if not stack or stack.pop() != mapping[char]:
                return False
        else:
            stack.append(char)
    return len(stack) == 0
```

2. Monotonic Stack (Next Greater Element)

```python
def next_greater(arr):
    n = len(arr)
    result = [-1] * n
    stack = []  # Stores indices
    for i in range(n):
        while stack and arr[i] > arr[stack[-1]]:
            result[stack.pop()] = arr[i]
        stack.append(i)
    return result
```

3. Largest Rectangle in Histogram

```python
def largest_rectangle(heights):
    stack = []
    max_area = 0
    heights.append(0)  # Sentinel
    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
        stack.append(i)
    return max_area
```

---

6. Queues

Fundamentals

- **FIFO:** First In, First Out
- Enqueue/Dequeue: O(1)

```
from collections import deque
queue = deque()
queue.append(x)      # Enqueue
queue.popleft()      # Dequeue
queue[0]             # Front
len(queue) == 0      # Is empty
```

Types

1. **Simple Queue**
2. **Circular Queue**
3. **Priority Queue (Heap)**
4. **Deque (Double-ended)**

Applications

1. **BFS Traversal**
2. **Level Order Traversal**
3. **Sliding Window Maximum**
4. **Task Scheduling**

Sliding Window Maximum (Monotonic Deque)

```
def max_sliding_window(nums, k):
    dq = deque()  # Stores indices
    result = []
    for i, num in enumerate(nums):
        # Remove elements outside window
        while dq and dq[0] < i - k + 1:
            dq.popleft()
        # Remove smaller elements
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

---

7. Hash Tables

Fundamentals

- **Key-Value pairs** with O(1) average operations
- **Collision handling:** Chaining, Open Addressing
- **Load Factor:** n/m (items/buckets)

```
# Dictionary
d = {}
d[key] = value       # Insert/Update O(1)
d.get(key, default) # Get with default
```

```
key in d            # Check existence O(1)
del d[key]          # Delete O(1)

# Set
s = set()
s.add(x)            # Add O(1)
s.remove(x)         # Remove O(1)
x in s              # Check O(1)
```

Applications

1. **Frequency counting**
2. **Two Sum pattern**
3. **Subarray sum problems**
4. **Detecting duplicates**
5. **Caching (LRU Cache)**

Two Sum

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

Subarray Sum Equals K

```
def subarray_sum(nums, k):
    count = 0
    prefix_sum = 0
    sum_count = {0: 1}
    for num in nums:
        prefix_sum += num
        if prefix_sum - k in sum_count:
            count += sum_count[prefix_sum - k]
        sum_count[prefix_sum] = sum_count.get(prefix_sum, 0) + 1
    return count
```

---

8. Trees

Binary Tree

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Tree Traversals

DFS Traversals

```python
# Inorder (Left, Root, Right) - BST gives sorted order
def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# Preorder (Root, Left, Right)
def preorder(root):
    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

# Postorder (Left, Right, Root)
def postorder(root):
    if not root:
        return []
    return postorder(root.left) + postorder(root.right) + [root.val]
```

Iterative Inorder

```python
def inorder_iterative(root):
    result, stack = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result
```

BFS (Level Order)

```python
def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

Binary Search Tree (BST)

Property: left < root < right for all nodes

```python
# Search in BST - O(log n) average, O(n) worst
def search(root, val):
    if not root or root.val == val:
        return root
    if val < root.val:
        return search(root.left, val)
    return search(root.right, val)

# Insert in BST
def insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root

# Validate BST
def is_valid_bst(root, min_val=float('-inf'), max_val=float('inf')):
    if not root:
        return True
    if root.val <= min_val or root.val >= max_val:
        return False
    return (is_valid_bst(root.left, min_val, root.val) and
            is_valid_bst(root.right, root.val, max_val))
```

Important Tree Algorithms

Height/Depth

```python
def height(root):
    if not root:
        return 0
    return 1 + max(height(root.left), height(root.right))
```

Lowest Common Ancestor

```python
def lca(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right:
        return root
    return left or right
```

Diameter of Tree

```python
def diameter(root):
    self.ans = 0
    def height(node):
        if not node:
            return 0
        left = height(node.left)
        right = height(node.right)
        self.ans = max(self.ans, left + right)
        return 1 + max(left, right)
```

```
    height(root)
    return self.ans
```

---

9. Heaps

Fundamentals

- **Min Heap:** Parent <= Children
- **Max Heap:** Parent >= Children
- **Complete Binary Tree** stored as array
- **Parent:** (i-1)//2, **Children:** 2i+1, 2i+2

```
import heapq

# Min heap (default in Python)
heap = []
heapq.heappush(heap, x)     # O(log n)
heapq.heappop(heap)         # O(log n)
heap[0]                     # Min element O(1)
heapq.heapify(arr)          # O(n)

# Max heap (negate values)
heapq.heappush(heap, -x)
-heapq.heappop(heap)
```

Applications

1. **Kth Largest/Smallest Element**
2. **Top K Elements**
3. **Merge K Sorted Lists**
4. **Median of Stream**
5. **Dijkstra's Algorithm**

Kth Largest Element

```
def find_kth_largest(nums, k):
    # Min heap of size k
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

Merge K Sorted Lists

```
def merge_k_lists(lists):
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst.val, i, lst))

    dummy = ListNode(0)
    curr = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
```

```
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Median from Data Stream

```
class MedianFinder:
    def __init__(self):
        self.small = []  # Max heap (negated)
        self.large = []  # Min heap

    def addNum(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2
```

---

10. Graphs

Representations

Adjacency List

```
# Unweighted
graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}

# Weighted
graph = {
    0: [(1, 5), (2, 3)],  # (neighbor, weight)
    1: [(0, 5), (3, 2)]
}

# Using defaultdict
from collections import defaultdict
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)  # For undirected
```

Adjacency Matrix

```
# n x n matrix, matrix[i][j] = 1 if edge exists
matrix = [[0] * n for _ in range(n)]
matrix[u][v] = 1
```

Graph Traversals

BFS

```python
def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return result
```

DFS

```python
def dfs(graph, start):
    visited = set()
    result = []

    def helper(node):
        visited.add(node)
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                helper(neighbor)

    helper(start)
    return result
```

Important Graph Algorithms

Topological Sort (Kahn's Algorithm)

```python
def topological_sort(n, edges):
    graph = defaultdict(list)
    indegree = [0] * n
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    queue = deque([i for i in range(n) if indegree[i] == 0])
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    return result if len(result) == n else []  # Empty if cycle
```

Cycle Detection (Directed Graph)

```python
def has_cycle(graph, n):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n

    def dfs(node):
        color[node] = GRAY
        for neighbor in graph[node]:
            if color[neighbor] == GRAY:  # Back edge
                return True
            if color[neighbor] == WHITE and dfs(neighbor):
                return True
        color[node] = BLACK
        return False

    return any(color[i] == WHITE and dfs(i) for i in range(n))
```

Dijkstra's Algorithm (Shortest Path)

```python
def dijkstra(graph, start, n):
    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)]

    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))

    return dist
```

Bellman-Ford (Negative Weights)

```python
def bellman_ford(edges, n, start):
    dist = [float('inf')] * n
    dist[start] = 0

    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    # Check negative cycle
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return None  # Negative cycle

    return dist
```

Number of Islands (Grid BFS/DFS)

```python
def num_islands(grid):
    if not grid:
```

```
        return 0

    rows, cols = len(grid), len(grid[0])
    count = 0

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == '0':
            return
        grid[r][c] = '0'  # Mark visited
        dfs(r+1, c)
        dfs(r-1, c)
        dfs(r, c+1)
        dfs(r, c-1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)

    return count
```

---

11. Sorting Algorithms

Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)	Yes

Implementations

Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
```

```
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Quick Sort

```
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

Counting Sort

```
def counting_sort(arr):
    if not arr:
        return arr

    max_val = max(arr)
    count = [0] * (max_val + 1)

    for num in arr:
        count[num] += 1

    result = []
    for i, c in enumerate(count):
        result.extend([i] * c)

    return result
```

---

12. Searching Algorithms

Binary Search

Prerequisite: Sorted array **Time:** O(log n)

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
```

```
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Binary Search Variants

```
# Find leftmost (first) occurrence
def find_left(arr, target):
    left, right = 0, len(arr)
    while left < right:
        mid = (left + right) // 2
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left

# Find rightmost (last) occurrence
def find_right(arr, target):
    left, right = 0, len(arr)
    while left < right:
        mid = (left + right) // 2
        if arr[mid] <= target:
            left = mid + 1
        else:
            right = mid
    return left - 1
```

Binary Search on Answer

```
# Example: Minimum capacity to ship packages
def ship_within_days(weights, days):
    def can_ship(capacity):
        current = 0
        day_count = 1
        for w in weights:
            if current + w > capacity:
                day_count += 1
                current = 0
            current += w
        return day_count <= days

    left, right = max(weights), sum(weights)
    while left < right:
        mid = (left + right) // 2
        if can_ship(mid):
            right = mid
        else:
            left = mid + 1
    return left
```

---

13. Recursion

Fundamentals

1. **Base Case:** Termination condition
2. **Recursive Case:** Problem breakdown
3. **Progress:** Move toward base case

```python
# Factorial
def factorial(n):
    if n <= 1:  # Base case
        return 1
    return n * factorial(n - 1)  # Recursive case

# Fibonacci
def fib(n, memo={}):
    if n <= 1:
        return n
    if n not in memo:
        memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

Recursion Patterns

1. Linear Recursion

```python
def sum_array(arr, n):
    if n == 0:
        return 0
    return arr[n-1] + sum_array(arr, n-1)
```

2. Binary Recursion (Divide & Conquer)

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

3. Tail Recursion

```python
def factorial_tail(n, acc=1):
    if n <= 1:
        return acc
    return factorial_tail(n - 1, n * acc)
```

---

14. Dynamic Programming

When to Use DP

1. **Optimal Substructure:** Optimal solution contains optimal solutions to subproblems
2. **Overlapping Subproblems:** Same subproblems solved multiple times

Approaches

Top-Down (Memoization)

```python
def fib_memo(n, memo={}):
    if n <= 1:
        return n
    if n not in memo:
        memo[n] = fib_memo(n-1) + fib_memo(n-2)
    return memo[n]
```

Bottom-Up (Tabulation)

```python
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

Classic DP Problems

Climbing Stairs

```python
def climb_stairs(n):
    if n <= 2:
        return n
    dp = [0] * (n + 1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

Coin Change

```python
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for i in range(coin, amount + 1):
            dp[i] = min(dp[i], dp[i - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Longest Common Subsequence

```python
def lcs(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

0/1 Knapsack

```python
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w],
                               dp[i-1][w - weights[i-1]] + values[i-1])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][capacity]
```

Longest Increasing Subsequence

```python
def lis(nums):
    if not nums:
        return 0

    # O(n^2) DP
    dp = [1] * len(nums)
    for i in range(1, len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

# O(n log n) with Binary Search
def lis_optimized(nums):
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

Edit Distance

```python
def edit_distance(word1, word2):
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
```

```
                dp[i][j] = 1 + min(dp[i-1][j],      # Delete
                                   dp[i][j-1],      # Insert
                                   dp[i-1][j-1])    # Replace

    return dp[m][n]
```

---

15. Greedy Algorithms

When to Use

- Local optimal choices lead to global optimal
- No need to reconsider previous decisions

Classic Problems

Activity Selection

```
def activity_selection(activities):
    # Sort by end time
    activities.sort(key=lambda x: x[1])
    result = [activities[0]]

    for i in range(1, len(activities)):
        if activities[i][0] >= result[-1][1]:
            result.append(activities[i])

    return result
```

Jump Game

```
def can_jump(nums):
    max_reach = 0
    for i, jump in enumerate(nums):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + jump)
    return True
```

Gas Station

```
def can_complete_circuit(gas, cost):
    total = current = start = 0
    for i in range(len(gas)):
        diff = gas[i] - cost[i]
        total += diff
        current += diff
        if current < 0:
            start = i + 1
            current = 0
    return start if total >= 0 else -1
```

---

16. Backtracking

Template

```
def backtrack(state, choices):
    if is_solution(state):
        add_to_result(state)
        return

    for choice in choices:
        if is_valid(choice):
            make_choice(state, choice)
            backtrack(state, choices)
            undo_choice(state, choice)  # Backtrack
```

Classic Problems

Subsets

```
def subsets(nums):
    result = []

    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return result
```

Permutations

```
def permutations(nums):
    result = []

    def backtrack(path, remaining):
        if not remaining:
            result.append(path[:])
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i+1:])
            path.pop()

    backtrack([], nums)
    return result
```

N-Queens

```
def solve_n_queens(n):
    result = []
    board = [['.'] * n for _ in range(n)]

    def is_safe(row, col):
        # Check column
        for i in range(row):
```

```
            if board[i][col] == 'Q':
                return False
        # Check diagonals
        for i, j in zip(range(row-1, -1, -1), range(col-1, -1, -1)):
            if board[i][j] == 'Q':
                return False
        for i, j in zip(range(row-1, -1, -1), range(col+1, n)):
            if board[i][j] == 'Q':
                return False
        return True

    def backtrack(row):
        if row == n:
            result.append([''.join(r) for r in board])
            return
        for col in range(n):
            if is_safe(row, col):
                board[row][col] = 'Q'
                backtrack(row + 1)
                board[row][col] = '.'

    backtrack(0)
    return result
```

---

17. Bit Manipulation

Basic Operations

Operation	Code	Description
AND	a & b	1 if both 1
OR	a | b	1 if any 1
XOR	a ^ b	1 if different
NOT	~a	Flip all bits
Left Shift	a « n	Multiply by 2^n
Right Shift	a » n	Divide by 2^n

Common Tricks

```
# Check if even/odd
is_odd = n & 1

# Check if power of 2
is_power_of_2 = n > 0 and (n & (n-1)) == 0

# Get ith bit
bit = (n >> i) & 1

# Set ith bit
n = n | (1 << i)

# Clear ith bit
n = n & ~(1 << i)
```

```python
# Toggle ith bit
n = n ^ (1 << i)

# Count set bits
def count_bits(n):
    count = 0
    while n:
        count += n & 1
        n >>= 1
    return count

# Brian Kernighan's (faster)
def count_bits_fast(n):
    count = 0
    while n:
        n &= (n - 1)  # Clear lowest set bit
        count += 1
    return count
```

Single Number (XOR)

```python
def single_number(nums):
    result = 0
    for num in nums:
        result ^= num
    return result
```

---

18. Two Pointers

Patterns

1. Opposite Direction

```python
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        current = arr[left] + arr[right]
        if current == target:
            return [left, right]
        elif current < target:
            left += 1
        else:
            right -= 1
    return []
```

2. Same Direction (Fast/Slow)

```python
def remove_duplicates(nums):
    if not nums:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1
```

3. Container With Most Water

```python
def max_area(heights):
    left, right = 0, len(heights) - 1
    max_water = 0
    while left < right:
        width = right - left
        height = min(heights[left], heights[right])
        max_water = max(max_water, width * height)
        if heights[left] < heights[right]:
            left += 1
        else:
            right -= 1
    return max_water
```

---

19. Sliding Window

Fixed Size Window

```python
def max_sum_subarray(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum

    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k]
        max_sum = max(max_sum, window_sum)

    return max_sum
```

Variable Size Window

```python
def min_subarray_len(target, nums):
    left = 0
    current_sum = 0
    min_len = float('inf')

    for right in range(len(nums)):
        current_sum += nums[right]
        while current_sum >= target:
            min_len = min(min_len, right - left + 1)
            current_sum -= nums[left]
            left += 1

    return min_len if min_len != float('inf') else 0
```

Longest Substring Without Repeating

```python
def length_of_longest_substring(s):
    char_index = {}
    left = max_len = 0

    for right, char in enumerate(s):
        if char in char_index and char_index[char] >= left:
            left = char_index[char] + 1
        char_index[char] = right
```

```
        max_len = max(max_len, right - left + 1)

    return max_len
```

---

20. Tries

Structure

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True

    def search(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end

    def starts_with(self, prefix):
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True
```

---

21. Union Find (Disjoint Set)

Structure

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # Path compression
```

```
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False
        # Union by rank
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True
```

Applications

- Connected components
- Cycle detection
- Kruskal's MST

---

22. Important Patterns

1. Prefix Sum

- Range sum queries
- Subarray sum problems

2. Monotonic Stack

- Next greater/smaller element
- Histogram problems

3. BFS for Shortest Path

- Unweighted graphs
- Level-order traversal

4. DFS for Exploration

- Path finding
- Connected components

5. Binary Search on Answer

- Optimization problems
- Minimize maximum / Maximize minimum

6. Interval Problems

- Sort by start or end
- Merge/count overlapping

7. Cycle Detection

- Floyd's (Linked List)
- Coloring (Graphs)

8. Tree DP

- Subtree problems
- Path problems

Quick Reference Card

Time Complexities

Data Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)	O(1)
Stack	O(n)	O(n)	O(1)	O(1)
Queue	O(n)	O(n)	O(1)	O(1)
Hash Table	-	O(1)	O(1)	O(1)
BST	O(log n)	O(log n)	O(log n)	O(log n)
Heap	-	O(n)	O(log n)	O(log n)

Algorithm Complexities

Algorithm	Time	Space
Binary Search	O(log n)	O(1)
BFS/DFS	O(V + E)	O(V)
Dijkstra	O((V+E) log V)	O(V)
Merge Sort	O(n log n)	O(n)
Quick Sort	O(n log n) avg	O(log n)

Document compiled for DSA interview preparation
Master these concepts to ace any product company interview!